

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Programmazione e Strutture Dati

Progetto d'Esame - Documentazione

Antonietta Santoro

NF12100104

A.A. 2025/2026

Sommario

1	Introduzione	1
2	Scelta ADT	2
3	Progettazione	5
4	Specifica Sintattica e Semantica	7
5	Test	18

1 Introduzione

Il progetto realizzato è un sistema di gestione delle segnalazioni in un comune.

Obiettivo: Gestire le segnalazioni inviate dai cittadini ad un comune in merito a problemi urbani. Il sistema deve consentire di registrare, classificare, monitorare ed aggiornare lo stato delle segnalazioni.

Ogni segnalazione deve contenere le seguenti informazioni:

- Codice identificativo
- Nome del cittadino
- Categoria del problema
- Descrizione
- Data di inserimento
- Livello di urgenza
- Stato della segnalazione

Le operazioni supportate devono essere:

- Registrazione di una nuova segnalazione
- Visualizzazione di tutte le segnalazioni registrate
- Ricerca di una segnalazione tramite codice o categoria
- Aggiornamento dello stato di una segnalazione
- Visualizzazione delle segnalazioni per stato
- Visualizzazione delle segnalazioni più urgenti
- Generazione di un report finale

Il progetto è stato realizzato in Linux.

2 Scelta ADT

2.1 Analisi del Problema

Considerando le funzionalità richieste e il contesto del problema si può analizzare la frequenza delle operazioni.

La registrazione di una nuova segnalazione è sicuramente un'operazione fondamentale del sistema, ma la sua frequenza dipende molto dalla grandezza del comune. La visualizzazione di tutte le segnalazioni è molto utile, ma la sua frequenza diminuirà all'aumentare delle segnalazioni registrate. L'utente favorirà, infatti, le operazioni di visualizzazione per stato e categoria. La generazione del report è un'operazione in genere non molto frequente.

Le restanti tre operazioni sono le più frequenti. La visualizzazione delle segnalazioni più urgenti è utile al comune per decidere di cosa occuparsi e ogni volta che una segnalazione viene risolta c'è bisogno di aggiornarne lo stato. L'operazione più frequente, però, è la ricerca tramite codice identificativo. Infatti, non solo questa è utilizzata come operazione a sé stante, ma viene anche richiamata dalla funzione di aggiornamento stato.

Le operazioni che quindi devono avere la maggiore velocità sono la ricerca e la visualizzazione delle segnalazioni più urgenti.

2.2 Confronto delle Strutture Dati

Per confrontare le diverse strutture dati ci concentriamo sull'operazione di ricerca. Prendiamo in considerazione le strutture che hanno minore complessità computazionale per questa operazione. Quindi le strutture da confrontare sono *BST*, *hashmap* e *heap* (nello specifico code a priorità).

ADT	Complessità Computazionale Ricerca	Note	Punto di Forza
BST	$O(\log n)$	-	Dati Ordinati
Hashmap	$O(1)$	In media, solo se gestite bene le collisioni. Nel caso peggiore (molto raro) diventa $O(n)$	Rapidità
Coda a priorità	$O(n)$	Solo nel caso di ricerca dell'elemento a priorità più alta diventa $O(1)$	Gestione delle priorità

Tabella 1 – Confronto operazione di ricerca in diversi ADT

Le due strutture che gestiscono meglio la ricerca per codice identificativo sono i BST e le hashmap. Gli alberi sono più lenti delle hashmap, ma hanno il vantaggio di supportare più operazioni come l'ordinamento dei dati. Per gestire le segnalazioni ad un comune, però, l'ordinamento dei dati non è essenziale. L'unico ordinamento utile in questo progetto è l'ordinamento per priorità per gestire al meglio la visualizzazione delle segnalazioni urgenti.

Analizziamo ora la complessità computazionale delle altre operazioni. Escludiamo la generazione report e l'aggiornamento dello stato di una segnalazione in quanto la prima è l'unione di altre operazioni e la seconda richiama la ricerca per codice identificativo.

In questa analisi consideriamo l'albero ordinato per codice identificativo.

Operazioni	BST	Hashmap	Coda a priorità
Registra Segnalazione (Inserimento)	$O(\log n)$ Se l'albero è bilanciato	$O(1)$	$O(\log n)$
Visualizza Tutte le Segnalazioni	$O(n)$	$O(n)$	$O(n)$
Ricerca per Codice Identificativo	$O(\log n)$	$O(1)^*$	$O(n)$
Ricerca per Categoria	$O(n)$	$O(n)$	$O(n)$
Visualizzazione per Stato	$O(n)$	$O(n)$	$O(n)$
Visualizzazione per <u>Priorità</u>	$O(n)$	$O(n)$	$O(1)$
*In media, solo se gestite bene le collisioni. Nel caso peggiore (molto raro) diventa $O(n)$			

Tabella 2 – Confronto delle operazioni negli ADT BST, Hashmap e Coda a Priorità

Nella tabella sono evidenziati in corsivo gli ADT con complessità computazionale minore rispetto alle operazioni. La visualizzazione completa, per stato e la ricerca per categoria hanno complessità computazionale $O(n)$ per tutte le strutture considerate perché comportano una visita completa della struttura.

È possibile avere un compromesso e utilizzare sia hashmap sia code a priorità per gestire le due operazioni più frequenti del progetto nel minor tempo possibile. Ed è stata questa la scelta finale nel progetto.

Un'altra importante considerazione da fare è sulla dinamicità delle strutture. Infatti, i BST sono dinamici mentre le code a priorità (implementate con array) e le hashmap non lo sono. Nel contesto del comune che riceve le segnalazioni, ogni comune può prevedere quale sia il numero

di segnalazioni che riceverà in base alla propria popolazione. Inoltre, scegliendo le corrette implementazioni per hashmap e coda a priorità queste possono contenere un numero variabile di dati andando a perdere efficienza.

Implementando una hashmap che gestisce le collisioni con il metodo di concatenazione si creano nell'effettivo delle liste, che quindi sono dinamiche. Quando però ci sono molte collisioni o il numero di elementi inseriti nella hashmap è superiore alla sua “dimensione massima”, cioè la dimensione dell'array di liste, la complessità computazionale della ricerca aumenta sempre più fino a raggiungere complessità $O(n)$. È poi possibile implementare un'operazione per ridimensionare la coda a priorità tramite *realloc*. In questo modo quando il numero di elementi inseriti aumenta oltre la dimensione massima il programma funzionerà comunque, anche se molto rallentato.

3 Progettazione

3.1 Interazione Utente

L'interazione utente avviene attraverso diversi menù sul terminale. Ogni funzione ha un numero a sé associato che l'utente può digitare per selezionarla. Il programma si occupa di pulire lo schermo del terminale ad ogni operazione compiuta.

3.2 Operazione di Eliminazione

Questo progetto non implementa l'operazione di eliminazione di una segnalazione. Si presuppone che tutte le segnalazioni inserite siano valide e che quindi quando vengono risolte il loro stato sia modificato in "CHIUSO". Queste sono ancora memorizzate nella hashmap e quindi ancora visualizzabili creando una "cronologia" di segnalazioni passate. Quando lo stato di una segnalazione viene aggiornato in "CHIUSO", questo non è più modificabile.

3.3 Interazione ADT

Gli ADT usati per il progetto sono: item (segnalazione), data, hashtable e priorityQueue.

L'*item* è un puntatore alla struttura *segnalazione* che è composta da: id, nome, categoria, descrizione, data, urgenza e stato. Questo rappresenta una segnalazione completa. Al momento della registrazione della segnalazione, l'utente inserisce nome, categoria, descrizione, data ed urgenza. Lo stato è sempre "APERTO", l'id viene generato automaticamente dal programma in base alla categoria e al numero già presente di segnalazioni in memoria.

La *data* è un puntatore alla struttura *dt* che è composta da giorno, mese ed anno. Questo ADT si occupa di verificare la validità delle date inserite dall'utente, di stamparle in formato gg/mm/aaaa e di confrontarle.

La *hashtable* è un puntatore alla struttura *hash* che è composta da: dimensione, numero di elementi e un array di *nodi*. La dimensione è la dimensione massima dell'array. Il numero di elementi è in realtà un array che contiene il numero di elementi divisi per categoria e il numero di elementi totali contenuti nella hashmap. L'array di nodi è l'effettiva struttura della hashmap. Ogni nodo è composto da un item segnalazione e da un puntatore al prossimo nodo. Questi sono la lista concatenata che gestisce le collisioni nella hashmap.

La *priorityQueue* è un puntatore alla struttura *PQ* che è composta da un array di item, il numero di elementi e la dimensione. Al momento della registrazione della segnalazione, questa viene inserita nella coda a priorità. Ogni volta che viene letta la segnalazione più urgente, si effettua un controllo per eliminare le segnalazioni più urgenti che hanno stato "CHIUSO" prima di restituire quella giusta. I due criteri di urgenza sono: il parametro *urgenza* della segnalazione e la data. Vengono considerate con maggiore priorità, a parità di *urgenza*, le segnalazioni con data più vecchia.

Inoltre, nel file *item.h* vengono definiti due tipi *enum*: categoria e stato. Questi sono usati per gestire più intuitivamente lo stato e categoria di ogni segnalazione. Per la categoria la

corrispondenza è: “*ILLUMINAZIONE = 0, GUASTI = 1, RIFIUTI = 2, STRADE = 3*”. Per lo stato la corrispondenza è: “*APERTO = 0, CHIUSO = 1, INLAVORAZIONE = 2*”.

Gli ADT sono tutti strettamente collegati. La hashmap è l’ADT principale di gestione delle segnalazioni, in quanto la maggior parte delle operazioni vengono effettuate attraverso essa. La priority queue invece gestisce solo l’operazione visualizzazione delle segnalazioni più urgenti. Entrambe utilizzano l’ADT item che a sua volta è implementato con i tipi *enum* e l’ADT *data*.

3.4 Operazioni Visualizza per Stato e Ricerca per Categoria

Queste due operazioni anche se molto simili sono state implementate diversamente.

L’operazione visualizza per stato è implementata in *hashmap.c* nella funzione “*stampa_Hashtable_stato*” facendo una visita completa dell’hashmap e controllando per ogni segnalazione se il suo stato sia quello richiesto dall’utente. Allo stesso modo si potrebbe implementare la funzione di ricerca per categoria, ma l’implementazione scelta è diversa.

Viste le particolari condizioni del sistema di non consentire l’eliminazione delle segnalazioni e della generazione dell’id in base al numero degli elementi già inseriti per categoria, viene riutilizzata l’operazione di ricerca per codice identificativo. Si generano prima tutti gli id della categoria selezionata dall’utente, conoscendone il numero, dopodiché si procede alla loro ricerca.

Questa implementazione diminuisce leggermente la complessità computazionale del problema: da $O(n)$, con n uguale al numero di elementi presenti nella hashmap, a $O(m)$ con $m \leq n$, il numero di elementi della categoria selezionata presenti nella hashmap.

Questo è valido se consideriamo la ricerca $O(1)$ nel caso medio. In caso delle condizioni non ottimali descritte precedentemente che rendono la ricerca $O(n)$, questa operazione è particolarmente inefficiente in quanto avrà complessità computazionale $O(n^2)$.

4 Specifica Sintattica e Semantica

4.1 ADT item

Specifica sintattica

Tipo di riferimento: item, categoria, stato

Tipi usati: categoria, stato, char*, int, data, FILE*

Operatori:

- crea_segna1azione(char*, char*, categoria, char*, data, int, stato) -> item
- libera_segna1azione(item) ->
- get_id(item) -> char*
- get_nome(item) -> char*
- get_cat(item) -> categoria
- get_descrizione(item) -> char*
- get_data(item) -> data
- get_urgenza(item) -> int
- get_stato(item) -> stato
- modifica_stato(item, stato) ->
- stampa_segna1azione(item) ->
- stampa_segna1azione_file(item, FILE*) ->
- confronta_segna1azioni(item, item) -> int

Specifica Semantica

Tipo di riferimento:

Il tipo item è l'insieme della 7-upla (id, nome, categoria, descrizione, data, urgenza e stato) i cui tipo sono rispettivamente char*, char*, categoria, char*, data, int, stato.

Categoria è un tipo enum: *ILLUMINAZIONE* = 0, *GUASTI* = 1, *RIFIUTI* = 2, *STRADE* = 3

Stato è un tipo enum: *APERTO* = 0, *CHIUSO* = 1, *INLAVORAZIONE* = 2

Operatori:

- crea_segna1azione(id, nome, cat, desc, data, u, st) -> s
 - Post: s = (id, nome, cat, desc, data, u, st)
- libera_segna1azione(s) ->
 - Post: s = NULL
- get_id(s) -> id
 - Post: s = (id, nome, cat, desc, data, u, st)
- get_nome(s) -> nome
 - Post: s = (id, nome, cat, desc, data, u, st)

- `get_cat(s) -> cat`
 - Post: `s = (id, nome, cat, desc, data, u, st)`
- `get_descrizione(s) -> desc`
 - Post: `s = (id, nome, cat, desc, data, u, st)`
- `get_data(s) -> data`
 - Post: `s = (id, nome, cat, desc, data, u, st)`
- `get_urgenza(s) -> u`
 - Post: `s = (id, nome, cat, desc, data, u, st)`
- `get_stato(s) -> st`
 - Post: `s = (id, nome, cat, desc, data, u, st)`
- `modifica_stato(s, st') ->`
 - Pre: `s = (id, nome, cat, desc, data, u, st)`
 - Post: `s = (id, nome, cat, desc, data, u, st')`
- `stampa_segna1azione(s) ->`
 - Pre: `s = (id, nome, cat, desc, data, u, st)`
 - Side Effect: Stampa a video di `s`
- `stampa_segna1azione_file(s, f) ->`
 - Pre: `s = (id, nome, cat, desc, data, u, st), f != NULL`
 - Side Effect: Stampa sul file `f` di `s`
- `confronta_segna1azioni(s1, s2) -> val`
 - Pre: `s1 = (id1, nome1, cat1, desc1, data1, u1, st1), s2 = (id2, nome2, cat2, desc2, data2, u2, st2)`
 - Post: `val = 0` se `id1 = id2, nome1 = nome2, cat1 = cat2, desc1 = desc2, data1 = data2, u1 = u2, st1 = st2`
`val = 1` altrimenti

4.2 ADT data

Specifica sintattica

Tipo di riferimento: data

Tipi usati: int, FILE*

Operatori:

- `crea_data(int, int, int) -> data`
- `libera_data(data) ->`
- `confronta_data(data, data) -> int`
- `valida_data(int, int, int) -> int`
- `stampa_data(data) ->`
- `stampa_data_file(data, FILE*) ->`

Specifica Semantica

Tipo di riferimento:

Il tipo data è l'insieme delle triple (giorno, mese, anno), il cui tipo è intero.

Operatori:

- crea_data(g, m, a) -> data
 - Post: data = (g, m, a)
- libera_data(data) ->
 - Post: data = NULL
- confronta_date(data1, data2) -> val
 - Pre: data1 = (g1, m1, a1), data2 = (g2, m2, a2)
 - Post: val = 1 se $(a1*10000 + m1*100 + g1) > (a2*10000 + m2*100 + g2)$;
val = 0 se $(a1*10000 + m1*100 + g1) = (a2*10000 + m2*100 + g2)$;
val = -1 se $(a1*10000 + m1*100 + g1) < (a2*10000 + m2*100 + g2)$.
- valida_data(g, m, a) -> val
 - Post: val = 1 se $a < 2000$ e $m < 1$ e $m > 12$ e $g < 1$ e $g < \text{numgMese}[m]$.
Dove numgMese = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}
- stampa_data(data) ->
 - Pre: data = (g, m, a)
 - Side Effect: Stampa a video di data
- stampa_data_file(data, f) ->
 - Pre: data = (g, m, a), f != NULL
 - Side Effect: Stampa sul file f di data

4.3 ADT hashtable

Specifica sintattica

Tipo di riferimento: hashtable, nodo

Tipi usati: item, int, char*, stato, FILE*

Operatori:

- crea_Hashtable(int) -> hashtable
- libera_Hashtable(hashtable) ->
- inserisci_Hash(hashtable, item) -> int
- ricerca(hashtable, char*) -> item
- get_numelem(hashtable, int) -> int
- get_dimensione(hashtable) -> int
- stampa_Hashtable(hashtable) -> int
- stampa_Hashtable_file(hashtable, FILE*) -> int
- stampa_Hashtable_stato(hashtable, stato) -> int
- stampa_Hashtable_stato_file(hashtable, stato, FILE*) -> int

Operatori ausiliari:

- hashFun(char*, int) -> int
- libera_lista(struct nodo) ->

Specifica Semantica

Tipo di riferimento:

Una hashtable è un insieme di elementi $H = \{a_1, a_2, \dots, a_n\}$ di tipo nodo. È formata anche da un intero dimensione e un array (num) che contiene il numero di elementi (item) divisi per categoria e il numero di elementi totali contenuti nella hashmap. In item il parametro id è la chiave della hashmap. Un nodo è formato da un puntatore ad item e un puntatore al prossimo nodo.

Operatori:

- crea_Hashtable(dim) -> h
 - Post: $h = \{\}$
 - Side Effect: $\text{num}[i] = 0, 0 \leq i \leq 4$
- libera_Hashtable(h) ->
 - Post: data = NULL
- inserisci_Hash(h, el) -> val
 - Post: val = -1 se $h = \text{NULL}$;
val = 0 se el è già presente in h;
val = 1 se inserimento riuscito.
 - Side Effect: dopo l'operazione $h = \{a_1, \dots, el, \dots, a_n\}$, con $n > 0$
 $\text{num}[4] = \text{num}[4] + 1, \text{num}[\text{item}(\text{categoria})] = \text{num}[\text{item}(\text{categoria})] + 1$
- ricerca(h, id) -> el
 - Pre: $h = \{a_1, a_2, \dots, a_n\}$, con $n > 0$
 - Post: se $a_i(\text{chiave}) = \text{id}$: $el = a_i$, con $1 \leq i \leq n$
Altrimenti $el = \text{NULLITEM}$
- get_numelem(h, m) -> numel
 - Pre: $h = \{a_1, a_2, \dots, a_n\}$, con $n > 0, 0 \leq m \leq 4$
 - Post: $\text{numel} = \text{num}[m]$
- get_dimensione(h) -> dim
 - Pre: $h = \{a_1, a_2, \dots, a_n\}$, con $n \geq 0$
 - Post: $\text{dim} = \text{dimensione}(h)$
- stampa_Hashtable(h) -> val
 - Pre: $h = \{a_1, a_2, \dots, a_n\}$, con $n > 0$
 - Post: val = 0 se $h = \text{NULL}$
val = 1 se la stampa va a buon fine
 - Side Effect: Stampa a video di h

- `stampa_Hashtable_file(h, f) -> val`
 - Pre: $h = \{a_1, a_2, \dots, a_n\}$, con $n > 0$, $f \neq \text{NULL}$
 - Post: $val = 0$ se $h = \text{NULL}$
 $val = 1$ se la stampa va a buon fine
 - Side Effect: Stampa sul file f di h
- `stampa_Hashtable_stato(h, st) -> val`
 - Pre: $h = \{a_1, a_2, \dots, a_n\}$, con $n > 0$, $0 \leq st \leq 2$
 - Post: $val = 0$ se $h = \text{NULL}$
 $val = 1$ se la stampa va a buon fine
 - Side Effect: Stampa a video di tutti gli item in h con stato st
- `stampa_Hashtable_stato_file(h, st, f) -> val`
 - Pre: $h = \{a_1, a_2, \dots, a_n\}$, con $n > 0$, $0 \leq st \leq 2$, $f \neq \text{NULL}$
 - Post: $val = 0$ se $h = \text{NULL}$
 $val = 1$ se la stampa va a buon fine
 - Side Effect: Stampa sul file f di tutti gli item in h con stato st

Operatori ausiliari:

- `hashFun(id, dim) -> index`
 - Post: $H_0 = 0$, $H(n) = [31 * H(n-1) + \text{ASCII}(\text{id}(n-1))] \bmod \text{dim}$, $0 \leq n \leq \text{dim}$
 $\text{index} = H(\text{dim})$
- `libera_lista(nodo) ->`
 - Post: $\text{nodo} = \text{NULL}$

4.4 ADT priorityQueue

Specifica sintattica

Tipo di riferimento: PQueue

Tipi usati: item, int

Operatori:

- `crea_PQ(int) -> PQueue`
- `libera_PQ(PQueue) ->`
- `ingrandisci_PQ(PQueue) -> int`
- `get_max_urgente(PQueue) -> item`
- `inserisci_PQ(PQueue, item) -> int`

Operatori ausiliari:

- `scendi(PQueue) ->`
- `sali(PQueue) ->`
- `vuota_PQ(PQueue) -> int`
- `deleteMax(PQueue) -> int`

Specifica Semantica

Tipo di riferimento:

PQueue = insieme delle code a priorità, dove: $\Lambda \in \text{PQueue}$ (coda vuota)

Esiste U , sottoinsieme dell'insieme degli item, non vuoto sul quale è definita una relazione d'ordine $\leq$ basata su urgenza e data di item.

PQueue è formata anche da due interi: dimensione e numero di elementi (num)

Operatori:

- crea_PQ(dim) -> pq
 - Post: pq = Λ
 - Side Effect: num = 0
- libera_PQ(pq) ->
 - Post: pq = NULL
- ingrandisci_PQ(pq) -> val
 - Post: val = 1 se l'operazione è riuscita
val = 0 altrimenti
 - Side Effect: dimensione(pq) = 2*dimensione(pq)
- get_max_urgente(pq) -> s
 - Post: s è la entry con la massima priorità fra quelle contenute in pq secondo la relazione $\leq$ sull'insieme U
Se pq = Λ , s = NULLITEM
 - Side Effect: Richiama la funzione deleteMax
- inserisci_PQ(pq, s) -> val
 - Post: val = 0 se pq = NULL o pq è piena
val = 1 se inserimento riuscito.
 - Side Effect: dopo l'operazione, se val = 1, pq contiene s
Richiama la funzione Sali
num = num + 1

Operatori ausiliari:

- scendi(pq) ->
 - Side Effect: pq è ordinata secondo la relazione $\leq$ sull'insieme U
- sali(pq) ->
 - Side Effect: pq è ordinata secondo la relazione $\leq$ sull'insieme U
- vuota_PQ(pq) -> val
 - Post: val = 1 se pq = Λ ,
val = 0 se pq $\neq \Lambda$

- deleteMax(pq) -> val
 - Pre: $pq \neq \Lambda$
 - Post: $val = 0$ se $pq = \text{NULL}$ o $pq = \Lambda$,
 $val = 1$ se la rimozione è riuscita.
 - Side Effect: dopo l'operazione, se $val = 1$, pq non contiene più la entry con la massima priorità secondo la relazione $\leq$ sull'insieme U
Richiama la funzione scendi
 $num = num - 1$

4.5 Utils

Specifica sintattica

Tipi usati: int, categoria, char*

Operatori:

- svuota_input_buffer() ->
- genera_id(categoria, int) -> char*
- valida_id(char*) -> int
- scambio_spazio_trattino(char*, char*) -> char*
- scambio_trattino_spazio(char*) ->

Specifica Semantica

Operatori:

- svuota_input_buffer() ->
 - Side Effect: Svuota il buffer dell'input
- genera_id(cat, num) -> id
 - Pre: $0 \leq cat \leq 3$, $num > 0$
 - Post: se $cat = \text{ILLUMINAZIONE}$ id = "ILL" concatenato STRINGA(num),
Se $cat = \text{GUASTI}$ id = "GUA" concatenato STRINGA(num),
Se $cat = \text{RIFIUTI}$ id = "RIF" concatenato STRINGA(num),
Se $cat = \text{STRADE}$ id = "STR" concatenato STRINGA(num)
- valida_id(id) -> val
 - Post: $val = 1$ se id è valido
 $val = 0$ se id non è valido
- scambio_spazio_trattino(str, n_str) -> n_str
 - Pre: $str = c_1 \dots s_1 \dots s_m \dots c_n$ con $n > 0$, $m \geq 0$, $s_i = ' '$ con $0 \leq i \leq m$,
 $n_str \neq \text{NULL}$
 - Post: $n_str = c_1 \dots t_1 \dots t_m \dots c_n$ con $n > 0$, $m \geq 0$, $t_i = '_'$ con $0 \leq i \leq m$
- scambio_trattino_spazio(str) ->
 - Pre: $str = c_1 \dots t_1 \dots t_m \dots c_n$ con $n > 0$, $m \geq 0$, $t_i = '_'$ con $0 \leq i \leq m$
 - Side Effect: $str = c_1 \dots s_1 \dots s_m \dots c_n$ con $n > 0$, $m \geq 0$, $s_i = ' '$ con $0 \leq i \leq m$,

4.6 Gestione

Specifica sintattica

Tipi usati: hashtable, PQueue, item, int, char*, categoria, stato

Operatori:

- stampa_menu() ->
- registra_segna_lazione_input(hashtable, PQueue) ->
- visualizza_segna_lazioni(hashtable) ->
- visualizza_segna_lazioni_stato(hashtable) ->
- ricerca_segna_lazione(hashtable) ->
- aggiorna_stato_segna_lazione(hashtable) ->
- visualizza_segna_lazione_urgente(PQueue) ->
- genera_report(hashtable) ->
- leggi_segna_lazioni_file(hashtable, PQueue) ->
- salva_segna_lazioni_file(hashtable) ->
- configura(hashtable*, PQueue*) ->

Operatori ausiliari:

- stampa_menu_ricerca() ->
- stampa_menu_visualizza_stato() ->
- stampa_intestazione_tabella() ->
- stampa_intestazione_tabella_file(FILE*) ->
- registra_segna_lazione(hashtable, PQueue, char*, categoria, char*, data, int, stato) -> int
- ricerca_id(hashtable, char*) -> int
- ricerca_categoria(hashtable, categoria) -> int
- visualizza_stato(hashtable, stato) -> int
- aggiorna_stato(hashtable, char*) -> int
- stampa_categoria_file(hashtable, categoria, int, FILE*) ->
- stampa_report_su_file(hashtable, char*) ->
- input_da_file(FILE*, hashtable*, PQueue*) ->
- output_file(FILE*, hashtable) ->

Specifica Semantica

Operatori:

- stampa_menu() ->
 - Side Effect: Stampa a video di un menu
- registra_segna_lazione_input(h, pq) ->
 - Side Effect: Interazione con l'utente
 - Richiama le funzioni: svuota_input_buffer, crea_data, registra_segna_lazione

- `visualizza_segnalazioni(h) ->`
 - Side Effect: Interazione con l'utente
Richiama le funzioni: `svuota_input_buffer`, `stampa_intestazione_tabella()`, `stampa_Hashtable`
- `visualizza_segnalazioni_stato(h) ->`
 - Side Effect: Interazione con l'utente
Richiama le funzioni: `svuota_input_buffer`, `stampa_menu_visualizza_stato`, `visualizza_stato`
- `ricerca_segnalazione(h) ->`
 - Side Effect: Interazione con l'utente
Richiama le funzioni: `svuota_input_buffer`, `stampa_menu_ricerca`, `valida_id`, `ricerca_id`, `ricerca_categoria`
- `aggiorna_stato_segnalazione(h) ->`
 - Side Effect: Interazione con l'utente
Richiama le funzioni: `svuota_input_buffer`, `valida_id`, `aggiorna_stato`
- `visualizza_segnalazione_urgente(pq) ->`
 - Side Effect: Interazione con l'utente
Richiama le funzioni: `svuota_input_buffer`, `get_max_urgente`, `stampa_intestazione_tabella`, `stampa_segnalazione`
- `genera_report(h) ->`
 - Side Effect: Creazione di un file "*report.txt*"
Richiama le funzioni: `svuota_input_buffer`, `stampa_report_su_file`
- `leggi_segnalazioni_file(h, pq) ->`
 - Side Effect: Stampa a video, interazione con l'utente, può sovrascrivere i dati
Richiama le funzioni: `svuota_input_buffer`, `libera_Hashtable`, `libera_PQ`, `input_da_file`
- `salva_segnalazioni_file(h) ->`
 - Side Effect: Stampa a video, interazione con l'utente, crea un file .txt
Richiama le funzioni: `svuota_input_buffer`, `output_file`
- `configura(h, pq) ->`
 - Side Effect: Sovrascrive i dati, `dimensione(h) = dimensione(pq)` viene modificata in base all'input dell'utente
Richiama le funzioni: `svuota_input_buffer`, `libera_Hashtable`, `libera_PQ`, `crea_Hashtable`, `crea_PQ`

Operatori ausiliari:

- `stampa_menu_ricerca() ->`
 - Side Effect: Stampa a video di un menu
- `stampa_menu_visualizza_stato() ->`
 - Side Effect: Stampa a video di un menu
- `stampa_intestazione_tabella() ->`
 - Side Effect: Stampa a video dell'intestazione di una tabella

- `stampa_intestazione_tabella_file(f) ->`
 - Side Effect: Stampa sul file f dell'intestazione di una tabella
- `registra_segna1azione(h, pq, nome, cat, desc, d, u, st) -> val`
 - Pre: $h \neq \text{NULL}$, $pq \neq \text{NULL}$, $\text{nome} \neq \text{NULL}$, $\text{desc} \neq \text{NULL}$, $d \neq \text{NULL}$, $0 \leq \text{cat} \leq 3$, $1 \leq u \leq 10$, $0 \leq \text{st} \leq 2$
 - Post: $\text{val} = \text{inserisci_Hash}(h, \text{segn})$, con $\text{segn} = (\text{id}, \text{nome}, \text{cat}, \text{desc}, d, u, \text{st})$
 - Side Effect: Richiama le funzioni: `get_numelem`, `genera_id`, `crea_segna1azione`, `inserisci_Hash`, `inserisci_PQ`
- `ricerca_id(h, ID) -> val`
 - Pre: id deve essere valido
 - Post: $\text{val} = 0$ se la segnalazione con $\text{id} = \text{ID}$ non è stata trovata
 $\text{val} = 1$ se la segnalazione con $\text{id} = \text{ID}$ è stata trovata
 - Side Effect: Richiama le funzioni: `stampa_intestazione_tabella`, `stampa_segna1azione`
- `ricerca_categoria(h, cat) -> val`
 - Post: $\text{val} = 0$ se non è stata trovata nessuna segnalazione con $\text{categoria} = \text{cat}$
 $\text{val} = 1$ se è stata trovata almeno una segnalazione con $\text{categoria} = \text{cat}$
 - Side Effect: Richiama le funzioni: `stampa_intestazione_tabella`, `stampa_segna1azione`, `genera_id`, `get_numelem`
- `visualizza_stato(h, st) -> val`
 - Post: $\text{val} = 1$ se la stampa delle segnalazioni con $\text{stato} = \text{st}$ avviene correttamente
 $\text{val} = 0$ se l'operazione fallisce
 - Side Effect: Richiama le funzioni: `stampa_intestazione_tabella`, `stampa_Hashtable_stato`
- `aggiorna_stato(h, ID) -> val`
 - Pre: id deve essere valido
 - Post: $\text{val} = 0$ se la segnalazione con $\text{id} = \text{ID}$ non è stata trovata
 $\text{val} = 1$ se l'operazione va a buon fine
 - Side Effect: Interazione con l'utente
Richiama le funzioni: `ricerca`, `get_stato`, `modifica_stato`
- `stampa_categoria_file(h, cat, num, f) ->`
 - Post: $\text{val} = 0$ se non è stata trovata nessuna segnalazione con $\text{categoria} = \text{cat}$
 $\text{val} = 1$ se è stata trovata almeno una segnalazione con $\text{categoria} = \text{cat}$
 - Side Effect: Richiama le funzioni: `stampa_intestazione_tabella_file`, `stampa_segna1azione_file`, `genera_id`, `ricerca`
- `stampa_report_su_file(h, fnome) ->`
 - Side Effect: Stampa report su file di nome `fnome`
Richiama le funzioni: `get_numelem`, `stampa_categoria_file`, `stampa_intestazione_tabella_file`, `stampa_Hashtable_stato_file`
- `input_da_file(f, h, pq) ->`
 - Side Effect: Sovrascrive h e q, inserisce segnalazioni in h e q presenti in f
Richiama le funzioni: `crea_Hashtable`, `crea_PQ`, `crea_data`, `scambio_trattino_spazio`, `scambio_trattino_spazio`, `crea_segna1azione`, `inserisci_Hash`, `inserisci_PQ`

- `output_file(f, h) ->`
 - Side Effect: Stampa su file `f` l'intera hashtable `h`
Richiama le funzioni: `get_dimensione`, `stampa_Hashtable_file`

5 Test

Sono stati svolti i seguenti test:

- Verifica della corretta registrazione delle segnalazioni
- Test della ricerca di segnalazioni per ID
- Test della ricerca di segnalazioni per categoria
- Verifica del corretto aggiornamento dello stato
- Test della gestione della priorità
- Verifica della visualizzazione per stato
- Test della generazione del report finale

5.1 Registrazione

Sono stati effettuati tre casi di test per la registrazione:

1. REG1 - Caso Base: Vengono fornite al programma tutte le informazioni che si aspetta per verificarne la correttezza.
2. REG2 - Registrazione Multipla: Vengono registrate due segnalazioni per verificare il corretto funzionamento del programma.
3. REG3 - Segnalazioni Uguali: Vengono passati due volte gli stessi parametri alla funzione di registrazione. Come previsto, il programma genera due ID diversi, e quindi due segnalazioni diverse.

Non sono stati realizzati casi in cui vengono forniti parametri non validi perché il programma sanitizza sempre l'input dell'utente.

5.2 Ricerca per ID

Sono stati effettuati tre casi di test per la ricerca per ID:

1. RID1 - Caso Base: Viene fornito un ID valido di una segnalazione presente nella hashmap.
2. RID2 - ID non Valido: Viene fornito un ID non valido, cioè di un formato diverso da quello previsto.
3. RID3 - Segnalazione non Esistente: Viene fornito un ID valido a cui però non corrisponde alcuna segnalazione nella hashmap.

5.3 Ricerca per Categoria

Sono stati effettuati due casi di test per la ricerca per categoria:

1. RCT1 - Caso Base: Viene fornita una categoria per la quale sono presenti segnalazioni nella hashmap.
2. RCT2 Nessuna Segnalazione per Categoria: Viene fornita una categoria per la quale non sono presenti segnalazioni nella hashmap.

Non è stato realizzato un caso in cui viene fornita una categoria non valida perché il programma sanitizza sempre l'input dell'utente.

5.4 Aggiorna Stato

Sono stati effettuati quattro casi di test per l'aggiornamento dello stato:

1. AGG1 Caso Base [APERTO]: Viene modificato lo stato di una segnalazione da APERTO a CHIUSO.
2. AGG2 Caso Base [IN LAVORAZIONE]: Viene modificato lo stato di una segnalazione da IN LAVORAZIONE ad APERTO.
3. AGG3 Aggiornamento non Supportato: Si prova a modificare lo stato di una segnalazione da CHIUSO ad APERTO. Il programma non consente di modificare lo stato una volta che è CHIUSO.
4. AGG4 Segnalazione non Esistente: Come ID della segnalazione da aggiornare, viene fornito un ID valido a cui però non corrisponde alcuna segnalazione nella hashmap.

Per semplicità non sono stati realizzati tutti i casi possibili, ma solo tre che figurano tutti gli stati possibili (APERTO -> CHIUSO; IN LAVORAZIONE -> APERTO; CHIUSO -> APERTO). Tutti gli altri casi hanno lo stesso comportamento di questi ultimi. Si può aggiornare liberamente dagli stati APERTO e IN LAVORAZIONE, mentre il programma non consente di aggiornare lo stato CHIUSO.

5.5 Gestione Priorità

Sono stati effettuati cinque casi di test per la gestione della priorità:

1. PRT1 Caso Base: Tutte le segnalazioni hanno parametro 'urgenza' diverso.
2. PRT2 Parametro 'urgenza' uguale: Due segnalazioni hanno il parametro 'urgenza' uguale. Si confrontano le date.
3. PRT3 Parametro 'urgenza' e data uguali: Due segnalazioni hanno il parametro 'urgenza' e 'data' uguale. In questo caso la segnalazione con priorità potrebbe essere qualunque delle due. Usando lo stesso input, però, la segnalazione con priorità sarà sempre la stessa.
4. PRT4 Segnalazione CHIUSA: La segnalazione a maggiore priorità è CHIUSA. Viene considerata la successiva.
5. PRT5 Tutte le Segnalazioni CHIUSE: Tutte le segnalazioni sono chiuse. La coda a priorità è vuota.

5.6 Visualizza per Stato

Sono stati effettuati quattro casi di test per la visualizzazione dello stato:

1. VST1 Caso Base [APERTO]: Visualizzazione delle segnalazioni con Stato APERTO.
2. VST2 Caso Base [CHIUSO]: Visualizzazione delle segnalazioni con Stato CHIUSO.
3. VST3 Caso Base [IN LAVORAZIONE]: Visualizzazione delle segnalazioni con Stato IN LAVORAZIONE.
4. VST4 Nessuna Segnalazione per Stato: Viene fornito uno stato per il quale non sono presenti segnalazioni nella hashmap.

Non è stato realizzato un caso in cui viene fornito uno stato non valido perché il programma sanitizza sempre l'input dell'utente.

5.7 Report

Sono stati effettuati due casi di test per la generazione del report:

1. REP1 Caso Base: Generazione di un report dopo un utilizzo standard del programma. Sono presenti segnalazioni di diverse categorie e con diversi stati.
2. REP2 Nessuna Segnalazione: Generazione di un report quando il programma non contiene nessuna segnalazione.